



✓ IEMOCAP Emotion Recognition — Modelling Pipeline

RoBERTa Text Classification

This notebook implements a **Text-Only Transformer pipeline** for 3-class Speech Emotion Recognition (SER) on the IEMOCAP corpus.

Component	Choice
Model	<code>roberta-base</code> fine-tuned for sequence classification
Split	Sessions 1–4 train · 10 % of train as val · Session 5 test (speaker-independent)
Loss	Weighted Cross-Entropy (compensates ~55 % Negative imbalance)
Primary metric	Macro F1-Score
Classes	<code>Negative</code> (0) · <code>Positive</code> (1) · <code>Neutral</code> (2)

Implementation Guidelines

Using a pre-trained RoBERTa model to classify transcription text into 3 emotion groups: **Neutral**, **Positive**, and **Negative**.

- Split: **Sessions 1–4 train / 10 % of train as validation / Session 5 held-out test** (speaker-independent)
- Address class imbalance with **inverse-frequency class weights** in the loss function
- Report **confusion matrix**, **accuracy**, and **Macro F1-Score**
- Use **early stopping** to prevent overfitting (patience = 3 epochs on val Macro F1)

✓ 1. Configuration & Imports

```
# — All hyperparameters in one place — change here, nowhere else —————
CONFIG = {
    "model_name":      "roberta-base",
    "text_column":     "transcription",
    "max_length":      128,
    "batch_size":      32,
    "lr":              2e-5,
    "weight_decay":    0.01,
    "epochs":          10,
    "early_stop_patience": 3,
    "test_session":    5,      # session held out as test set — matches audio/fusion setup
    "val_ratio":        0.10,  # fraction of Ses01-04 used for validation
    "random_seed":     42,
    "num_labels":      3,
}

LABEL2ID = {"Negative": 0, "Positive": 1, "Neutral": 2}
ID2LABEL = {v: k for k, v in LABEL2ID.items()}

EMOTION_GROUP_MAP = {
    "happy":      "Positive",
    "excited":    "Positive",
    "angry":      "Negative",
    "frustrated": "Negative",
    "fear":       "Negative",
    "sad":        "Negative",
    "disgust":    "Negative",
    "other":      "Neutral",
    "neutral":    "Neutral",
    "surprise":   "Neutral",
}

import os
import sys
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from torch.utils.data import Dataset, DataLoader
```

```

from transformers import (
    RobertaTokenizer,
    RobertaForSequenceClassification,
    get_linear_schedule_with_warmup,
)
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    f1_score, accuracy_score,
    confusion_matrix, classification_report,
)
from tqdm import tqdm

torch.manual_seed(CONFIG["random_seed"])
np.random.seed(CONFIG["random_seed"])

# — Device — CUDA > MPS (Apple Silicon) > CPU —————
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
print(f"Device : {device}")
print(f"PyTorch: {torch.__version__}")

# — Checkpoint directory — Google Drive in Colab, ~/Downloads locally —————
IN_COLAB = "google.colab" in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')
    CHECKPOINT_DIR = "/content/drive/MyDrive"
else:
    CHECKPOINT_DIR = os.path.expanduser("~/Downloads")

Device : cuda
PyTorch: 2.11.0+cu128
Mounted at /content/drive

```

2. Data Loading & Preprocessing

```

from datasets import load_dataset

def load_and_preprocess(
    emotion_map: dict,
    label2id: dict,
    text_col: str,
) -> pd.DataFrame:
    """Load IEMOCAP from HuggingFace, group emotions, encode labels, drop unknowns.

    Returns a DataFrame with columns: text_col, emotion_group, label, session_num.
    """
    ds = load_dataset("AbstractTTS/IEMOCAP")
    df = ds["train"].to_pandas()

    df["emotion_group"] = df["major_emotion"].map(emotion_map)
    df = df.dropna(subset=["emotion_group", text_col]).reset_index(drop=True)
    df["label"] = df["emotion_group"].map(label2id)
    df["session_num"] = df["file"].str.extract(r"Ses\d{2}").astype(int)

    print(f"Total samples after cleaning: {len(df)}")
    print(df["emotion_group"].value_counts().to_string())
    return df

def loso_split(
    df: pd.DataFrame,
    test_session: int,
    val_ratio: float,
    seed: int,
) -> tuple:
    """Hold out test_session as test; split the remaining sessions into train/val.

    Returns: (train_df, val_df, test_df)
    """
    test_df = df[df["session_num"] == test_session].reset_index(drop=True)
    trainval_df = df[df["session_num"] != test_session].reset_index(drop=True)

```

```

train_df, val_df = train_test_split(
    trainval_df,
    test_size=val_ratio,
    stratify=trainval_df["label"],
    random_state=seed,
)

for name, part in [("Train", train_df), ("Val", val_df), ("Test", test_df)]:
    counts = " | ".join(
        f"{k}: {v}" for k, v in part["emotion_group"].value_counts().items()
    )
    print(f"{name:5s}: {len(part):5d} rows | {counts}")

return (
    train_df.reset_index(drop=True),
    val_df.reset_index(drop=True),
    test_df.reset_index(drop=True),
)

df = load_and_preprocess(EMOTION_GROUP_MAP, LABEL2ID, CONFIG["text_column"])
train_df, val_df, test_df = loso_split(
    df, CONFIG["test_session"], CONFIG["val_ratio"], CONFIG["random_seed"]
)

```

```

README.md: 100%                                1.07k/1.07k [00:00<00:00, 73.0kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
data/train-00000-of-00003.parquet: 100%          489M/489M [00:05<00:00, 102MB/s]
data/train-00001-of-00003.parquet: 100%          456M/456M [00:07<00:00, 71.0MB/s]
data/train-00002-of-00003.parquet: 100%          462M/462M [00:10<00:00, 46.0MB/s]
Generating train split: 100%                      10039/10039 [00:09<00:00, 1279.83 examples/s]
Total samples after cleaning: 10039
emotion_group
Negative    5545
Positive    2632
Neutral     1862
Train: 7082 rows | Negative: 3956 | Positive: 1817 | Neutral: 1309
Val : 787 rows | Negative: 440 | Positive: 202 | Neutral: 145
Test : 2170 rows | Negative: 1149 | Positive: 613 | Neutral: 408

```

```

# === Sum-then-Argmax: corrected ground truth + human-consensus score ===
group_emotions = {
    'Positive': ['happy', 'excited'],
    'Negative': ['angry', 'frustrated', 'fear', 'sad', 'disgust'],
    'Neutral': ['neutral', 'surprise'],
}
for group, cols in group_emotions.items():
    df[f'{group}_prob'] = df[cols].sum(axis=1)

prob_cols = [f'{g}_prob' for g in group_emotions]
df['Corrected_Ground_Truth'] = df[prob_cols].idxmax(axis=1).str.replace('_prob', '', regex=False)
df['consensus'] = df[prob_cols].max(axis=1)
df['corrected_label'] = df['Corrected_Ground_Truth'].map(LABEL2ID)

# Re-split so train/val/test carry the new columns (same seed -> identical partition)
train_df, val_df, test_df = loso_split(
    df, CONFIG["test_session"], CONFIG["val_ratio"], CONFIG["random_seed"]
)

print("Corrected_Ground_Truth distribution:")
print(df['Corrected_Ground_Truth'].value_counts().to_string())
print(f"\nMean human consensus: {df['consensus'].mean():.4f}")

n_flipped = (df['emotion_group'] != df['Corrected_Ground_Truth']).sum()
print(f"Labels flipped vs. major_emotion grouping: {n_flipped} / {len(df)} ({n_flipped/len(df)*100:.2f}%)")

```

```

Train: 7082 rows | Negative: 3956 | Positive: 1817 | Neutral: 1309
Val : 787 rows | Negative: 440 | Positive: 202 | Neutral: 145
Test : 2170 rows | Negative: 1149 | Positive: 613 | Neutral: 408
Corrected_Ground_Truth distribution:
Corrected_Ground_Truth
Negative    5721
Positive    2504
Neutral     1814

Mean human consensus: 0.8055
Labels flipped vs. major_emotion grouping: 503 / 10039 (5.01%)

```

3. Dataset & DataLoaders

```
class IEMOCAPTextDataset(Dataset):
    """PyTorch Dataset that tokenises transcription text for RoBERTa fine-tuning."""

    def __init__(
        self,
        texts: list,
        labels: list,
        tokenizer,
        max_length: int,
    ):
        self.encodings = tokenizer(
            texts,
            truncation=True,
            padding="max_length",
            max_length=max_length,
            return_tensors="pt",
        )
        self.labels = torch.tensor(labels, dtype=torch.long)

    def __len__(self) -> int:
        return len(self.labels)

    def __getitem__(self, idx: int) -> dict:
        return {
            "input_ids": self.encodings["input_ids"][idx],
            "attention_mask": self.encodings["attention_mask"][idx],
            "labels": self.labels[idx],
        }

def build_data loaders(
    train_df: pd.DataFrame,
    val_df: pd.DataFrame,
    test_df: pd.DataFrame,
    tokenizer,
    config: dict,
) -> tuple:
    """Tokenise each split and wrap in DataLoaders.

    Returns: (train_loader, val_loader, test_loader)
    """
    text_col = config["text_column"]
    max_length = config["max_length"]
    batch_size = config["batch_size"]

    train_ds = IEMOCAPTextDataset(
        train_df[text_col].tolist(), train_df["label"].tolist(), tokenizer, max_length
    )
    val_ds = IEMOCAPTextDataset(
        val_df[text_col].tolist(), val_df["label"].tolist(), tokenizer, max_length
    )
    test_ds = IEMOCAPTextDataset(
        test_df[text_col].tolist(), test_df["label"].tolist(), tokenizer, max_length
    )

    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False)
    test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)
    return train_loader, val_loader, test_loader

tokenizer = RobertaTokenizer.from_pretrained(CONFIG["model_name"])
train_loader, val_loader, test_loader = build_data loaders(
    train_df, val_df, test_df, tokenizer, CONFIG
)
print(f"Batches - train: {len(train_loader)} | val: {len(val_loader)} | test: {len(test_loader)}")
```

```
tokenizer_config.json: 100% 25.0/25.0 [00:00<00:00, 2.62kB/s]
vocab.json: 100% 899k/899k [00:00<00:00, 6.55MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 5.54MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 7.32MB/s]
Batches - train: 222 | val: 25 | test: 68
```

4. Model, Class Weights & Optimiser

```
def compute_weights(train_labels: list, num_labels: int) -> torch.Tensor:
    """Compute inverse-frequency class weights from the training set labels."""
    classes = np.arange(num_labels)
    weights = compute_class_weight("balanced", classes=classes, y=train_labels)
    return torch.tensor(weights, dtype=torch.float)

def build_model(config: dict, id2label: dict, label2id: dict):
    """Load roberta-base with a 3-class sequence classification head."""
    model = RobertaForSequenceClassification.from_pretrained(
        config["model_name"],
        num_labels=config["num_labels"],
        id2label=id2label,
        label2id=label2id,
    )
    return model.to(device)

class_weights = compute_weights(
    train_df["label"].tolist(), CONFIG["num_labels"]
).to(device)
print(
    "Class weights:",
    {ID2LABEL[i]: f"{w:.3f}" for i, w in enumerate(class_weights.cpu().tolist())},
)

model = build_model(CONFIG, ID2LABEL, LABEL2ID)
criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=CONFIG["lr"], weight_decay=CONFIG["weight_decay"]
)

total_steps = len(train_loader) * CONFIG["epochs"]
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=total_steps // 10,
    num_training_steps=total_steps,
)
print(f"Total optimisation steps: {total_steps}")
```

Class weights: {'Negative': '0.597', 'Positive': '1.299', 'Neutral': '1.803'}

config.json: 100% 481/481 [00:00<00:00, 48.9kB/s]

model.safetensors: 100% 499M/499M [00:08<00:00, 59.8MB/s]

Loading weights: 100% 197/197 [00:00<00:00, 1931.66it/s]

[transformers] **RobertaForSequenceClassification** LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.bias	MISSING
classifier.out_proj.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your dow

Total optimisation steps: 2220

5. Training Loop with Early Stopping

```
def train_epoch(
    model,
    loader: DataLoader,
    optimizer,
    scheduler,
    criterion: nn.Module,
    device: torch.device,
) -> float:
    """Run one training epoch. Returns mean cross-entropy loss over all batches."""
    model.train()
```

```

total_loss = 0.0

for batch in tqdm(loader, desc="  train", leave=False):
    input_ids      = batch["input_ids"].to(device)
    attention_mask = batch["attention_mask"].to(device)
    labels         = batch["labels"].to(device)

    optimizer.zero_grad()
    logits = model(input_ids=input_ids, attention_mask=attention_mask).logits
    loss    = criterion(logits, labels)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    scheduler.step()
    total_loss += loss.item()

return total_loss / len(loader)

def evaluate(
    model,
    loader: DataLoader,
    criterion: nn.Module,
    device: torch.device,
) -> dict:
    """Evaluate model on a DataLoader. Returns dict with loss, accuracy, macro_f1."""
    model.eval()
    total_loss, all_preds, all_labels = 0.0, [], []

    with torch.no_grad():
        for batch in loader:
            input_ids      = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels         = batch["labels"].to(device)

            logits = model(input_ids=input_ids, attention_mask=attention_mask).logits
            total_loss += criterion(logits, labels).item()
            all_preds.extend(logits.argmax(dim=-1).cpu().tolist())
            all_labels.extend(labels.cpu().tolist())

    return {
        "loss":      total_loss / len(loader),
        "accuracy":  accuracy_score(all_labels, all_preds),
        "macro_f1":  f1_score(all_labels, all_preds, average="macro"),
    }

def fit(
    model,
    train_loader: DataLoader,
    val_loader:   DataLoader,
    optimizer,
    scheduler,
    criterion:    nn.Module,
    config:       dict,
    device:       torch.device,
) -> dict:
    """Full training loop with early stopping on val Macro F1.

    Restores the best checkpoint at the end.
    Returns history dict with per-epoch train/val loss and F1.
    """
    best_f1, patience_count = 0.0, 0
    best_state = None
    history = {"train_loss": [], "val_loss": [], "train_f1": [], "val_f1": []}

    for epoch in range(1, config["epochs"] + 1):
        train_loss = train_epoch(model, train_loader, optimizer, scheduler, criterion, device)
        train_metrics = evaluate(model, train_loader, criterion, device)
        val_metrics   = evaluate(model, val_loader,   criterion, device)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_metrics["loss"])
        history["train_f1"].append(train_metrics["macro_f1"])
        history["val_f1"].append(val_metrics["macro_f1"])

        print(
            f"Epoch {epoch:02d}/{config['epochs']} | "
            f"train loss {train_loss:.4f} | "
            f"val loss {val_metrics['loss']:.4f} | "
            f"val acc {val_metrics['accuracy']:.4f} | "
            f"val F1 {val_metrics['macro_f1']:.4f}"

```

```

    if val_metrics["macro_f1"] > best_f1:
        best_f1 = val_metrics["macro_f1"]
        best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        patience_count = 0
    else:
        patience_count += 1
        if patience_count >= config["early_stop_patience"]:
            print(
                f"Early stopping at epoch {epoch} "
                f"(no improvement for {config['early_stop_patience']} epochs)"
            )
            break

model.load_state_dict({k: v.to(device) for k, v in best_state.items()})
print(f"\nBest val Macro F1: {best_f1:.4f} – best checkpoint restored.")
return history

```

6. Run Training

```

history = fit(
    model, train_loader, val_loader,
    optimizer, scheduler, criterion,
    CONFIG, device,
)

```

```

Epoch 01/10 | train loss 1.0614 | val loss 0.8693 | val acc 0.6404 | val F1 0.6030
Epoch 02/10 | train loss 0.8459 | val loss 0.8135 | val acc 0.6290 | val F1 0.6136
Epoch 03/10 | train loss 0.6818 | val loss 0.7628 | val acc 0.6925 | val F1 0.6601
Epoch 04/10 | train loss 0.5811 | val loss 0.7613 | val acc 0.7192 | val F1 0.6827
Epoch 05/10 | train loss 0.4819 | val loss 0.8295 | val acc 0.7027 | val F1 0.6691
Epoch 06/10 | train loss 0.4286 | val loss 0.8208 | val acc 0.6925 | val F1 0.6678
Epoch 07/10 | train loss 0.3682 | val loss 1.0039 | val acc 0.7332 | val F1 0.6871
Epoch 08/10 | train loss 0.3274 | val loss 1.0843 | val acc 0.7332 | val F1 0.6740
Epoch 09/10 | train loss 0.2991 | val loss 1.1133 | val acc 0.7230 | val F1 0.6730
Epoch 10/10 | train loss 0.2698 | val loss 1.1787 | val acc 0.7243 | val F1 0.6654
Early stopping at epoch 10 (no improvement for 3 epochs)

```

Best val Macro F1: 0.6871 – best checkpoint restored.

Save Model Weights

To save the trained model's weights, we can use `torch.save()` to store the `state_dict()` of the model. This will create a `.pth` file containing the learned parameters.

```

model_save_path = "./best_model_weights.pth"
try:
    torch.save(model.state_dict(), model_save_path)
    print(f"Model weights saved to {model_save_path}")
except NameError:
    print("Error: 'model' or 'torch' not defined. Please ensure previous cells have been run.")

```

Model weights saved to ./best_model_weights.pth

Load the Model and Make Predictions

After saving the model weights, you can load the model back into memory and use it for inference on new data. This involves:

1. Re-initializing the model with the same architecture.
2. Loading the saved `state_dict` into the model.
3. Tokenizing a new input text.
4. Passing the tokenized input through the model to get predictions.

```

# Re-initialize the model with the same architecture
try:
    loaded_model = build_model(CONFIG, ID2LABEL, LABEL2ID)

    # Load the saved state dictionary
    loaded_model.load_state_dict(torch.load(model_save_path))
    loaded_model.to(device)
    loaded_model.eval() # Set the model to evaluation mode

    print("Model loaded successfully!")
except NameError:

```

```
print("Error: 'build_model', 'CONFIG', 'ID2LABEL', 'LABEL2ID', 'device', or 'torch' not defined. Ensure prev
except FileNotFoundError:
    print(f"Error: Model weights not found at {model_save_path}. Please ensure the saving cell was executed.")
```

Loading weights: 100% 197/197 [00:00<00:00, 4665.21it/s]

[transformers] **RobertaForSequenceClassification** LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.bias	MISSING
classifier.out_proj.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your own data.

Model loaded successfully!

```
def predict_emotion(text: str, model, tokenizer, device, id2label: dict, max_length: int = 128):
    """Predicts the emotion of a given text using the loaded model."""
    # Tokenize the input text
    inputs = tokenizer(
        text,
        truncation=True,
        padding='max_length',
        max_length=max_length,
        return_tensors='pt'
    )

    # Move inputs to the correct device
    input_ids = inputs['input_ids'].to(device)
    attention_mask = inputs['attention_mask'].to(device)

    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits

    # Get the predicted label ID and map it back to the emotion name
    predicted_label_id = torch.argmax(logits, dim=-1).item()
    predicted_emotion = id2label[predicted_label_id]

    return predicted_emotion

# Example usage:
try:
    new_text = "I am so happy to hear that!"
    predicted_emotion = predict_emotion(new_text, loaded_model, tokenizer, device, ID2LABEL, CONFIG['max_length'])
    print(f"Text: '{new_text}'\nPredicted Emotion: {predicted_emotion}")

    new_text_2 = "This is absolutely infuriating."
    predicted_emotion_2 = predict_emotion(new_text_2, loaded_model, tokenizer, device, ID2LABEL, CONFIG['max_length'])
    print(f"Text: '{new_text_2}'\nPredicted Emotion: {predicted_emotion_2}")

    new_text_3 = "I am going to the store now."
    predicted_emotion_3 = predict_emotion(new_text_3, loaded_model, tokenizer, device, ID2LABEL, CONFIG['max_length'])
    print(f"Text: '{new_text_3}'\nPredicted Emotion: {predicted_emotion_3}")
except NameError:
    print("Error: 'loaded_model', 'tokenizer', 'device', 'ID2LABEL', or 'CONFIG' not defined. Ensure model loadi
```

Text: 'I am so happy to hear that!'
Predicted Emotion: Positive

Text: 'This is absolutely infuriating.'
Predicted Emotion: Negative

Text: 'I am going to the store now.'
Predicted Emotion: Negative

Save Model Weights

To save the trained model's weights, we can use `torch.save()` to store the `state_dict()` of the model. This will create a `.pth` file containing the learned parameters.

```
model_save_path = "./best_model_weights.pth"
torch.save(model.state_dict(), model_save_path)
```



```
print(f"Model weights saved to {model_save_path}")
```

```
Model weights saved to ./best_model_weights.pth
```

```
model_save_path_pt = os.path.join(CHECKPOINT_DIR, "text_best_model_weights.pt")
torch.save(model.state_dict(), model_save_path_pt)
print(f"Model weights saved to {model_save_path_pt}")
```

```
Model weights saved to /content/drive/MyDrive/text_best_model_weights.pt
```

7. Final Evaluation on Test Set

```
def plot_confusion_matrix(cm: np.ndarray, class_names: list) -> None:
    """Display a labelled confusion matrix heatmap."""
    fig, ax = plt.subplots(figsize=(6, 5))
    sns.heatmap(
        cm, annot=True, fmt=".2f", cmap="Blues",
        xticklabels=class_names, yticklabels=class_names,
        ax=ax,
    )
    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    ax.set_title("Confusion Matrix - Test Set")
    plt.tight_layout()
    plt.show()

def report_metrics(
    model,
    test_loader: DataLoader,
    device: torch.device,
    id2label: dict,
) -> None:
    """Print full classification report and confusion matrix for the test set."""
    model.eval()
    all_preds, all_labels = [], []

    with torch.no_grad():
        for batch in test_loader:
            logits = model(
                input_ids=batch["input_ids"].to(device),
                attention_mask=batch["attention_mask"].to(device),
            ).logits
            all_preds.extend(logits.argmax(dim=-1).cpu().tolist())
            all_labels.extend(batch["labels"].tolist())

    class_names = [id2label[i] for i in sorted(id2label)]

    print("=" * 50)
    print(f"Test Accuracy : {accuracy_score(all_labels, all_preds):.4f}")
    print(f"Test Macro F1 : {f1_score(all_labels, all_preds, average='macro'):.4f}")
    print("=" * 50)
    print(classification_report(all_labels, all_preds, target_names=class_names))

    cm = confusion_matrix(all_labels, all_preds, normalize='true')
    plot_confusion_matrix(cm, class_names)

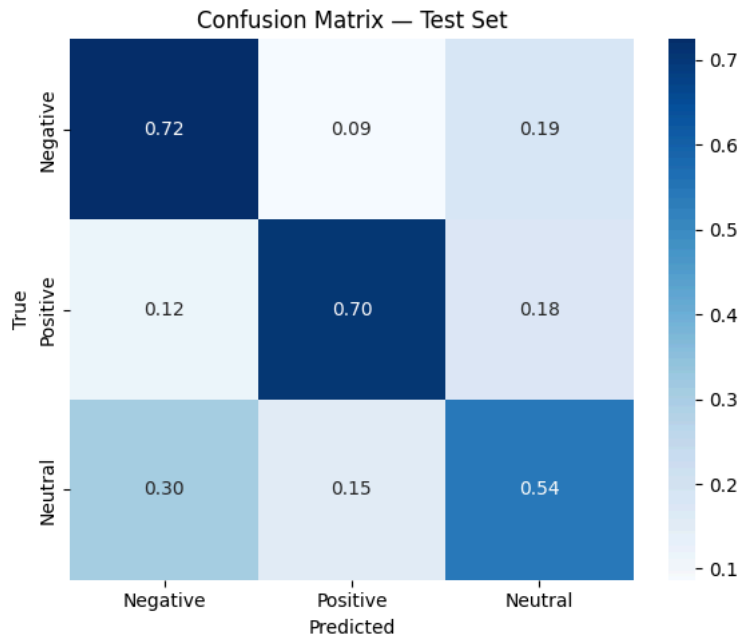
report_metrics(model, test_loader, device, ID2LABEL)
```

```

=====
Test Accuracy : 0.6843
Test Macro F1 : 0.6475
=====

```

	precision	recall	f1-score	support
Negative	0.81	0.72	0.76	1149
Positive	0.73	0.70	0.72	613
Neutral	0.40	0.54	0.46	408
accuracy			0.68	2170
macro avg	0.65	0.66	0.65	2170
weighted avg	0.71	0.68	0.69	2170



8. Training Curves

```

def plot_training_curves(history: dict) -> None:
    """Plot train/val loss and Macro F1 over epochs side by side."""
    epochs = range(1, len(history["train_loss"]) + 1)
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.plot(epochs, history["train_loss"], label="Train")
    ax1.plot(epochs, history["val_loss"], label="Val")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Cross-Entropy Loss")
    ax1.set_title("Loss Curves")
    ax1.legend()

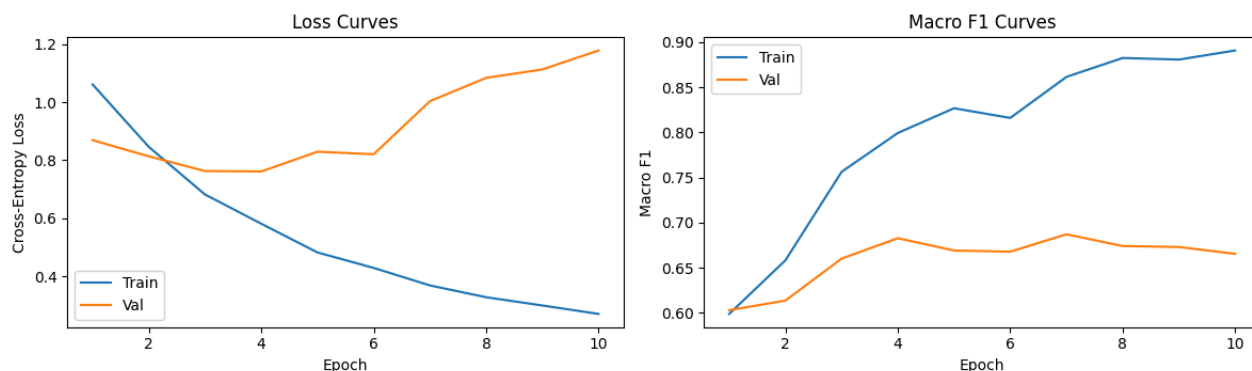
    ax2.plot(epochs, history["train_f1"], label="Train")
    ax2.plot(epochs, history["val_f1"], label="Val")
    ax2.set_xlabel("Epoch")
    ax2.set_ylabel("Macro F1")
    ax2.set_title("Macro F1 Curves")
    ax2.legend()

    plt.suptitle("RoBERTa Text Classifier – Training History", y=1.02)
    plt.tight_layout()
    plt.show()

plot_training_curves(history)

```

RoBERTa Text Classifier — Training History



9. Corrected Ground-Truth Evaluation (Sum-then-Argmax)

Re-evaluates the **already-trained** checkpoint against a ground truth built by summing the 9 raw soft-label probabilities into Negative/Positive/Neutral *before* taking the argmax (rather than grouping the `major_emotion` hard label after the fact). No retraining — this reloads `text_best_model_weights.pt` and runs a forward pass only. Also reports the Consensus Tier Table: accuracy/F1 stratified by how much annotators agreed on each utterance's grouped label.

```
def corrected_eval_report(test_df, y_pred, model_name="Model"):
    """Compare old vs. corrected Macro F1, and break down performance by human-consensus tier."""
    y_pred = np.array(y_pred)
    y_true_old = test_df['label'].values
    y_true_corrected = test_df['corrected_label'].values

    old_f1 = f1_score(y_true_old, y_pred, average='macro')
    new_f1 = f1_score(y_true_corrected, y_pred, average='macro')
    print(f"=== {model_name} ===")
    print(f"Old Macro F1 (major_emotion GT) : {old_f1:.4f}")
    print(f"Corrected Macro F1 (Sum-then-Argmax GT) : {new_f1:.4f} (Δ {new_f1 - old_f1:+.4f})")

    bins = [0, 0.5, 0.75, 1.0]
    tier_labels = ['Low (<0.5)', 'Medium (0.5-0.75)', 'High (>0.75)']
    tiers = pd.cut(test_df['consensus'], bins=bins, labels=tier_labels, include_lowest=True)

    rows = []
    for tier in tier_labels:
        mask = (tiers == tier).values
        if mask.sum() == 0:
            continue
        rows.append({
            'Tier': tier, 'N': int(mask.sum()),
            'Accuracy': accuracy_score(y_true_corrected[mask], y_pred[mask]),
            'Macro F1': f1_score(y_true_corrected[mask], y_pred[mask], average='macro'),
        })
    tier_table = pd.DataFrame(rows)
    print(tier_table.to_string(index=False))
    print()
    return old_f1, new_f1, tier_table
```

```
# No retraining: reload the saved checkpoint fresh and run a forward pass only.
eval_ckpt_path = os.path.join(CHECKPOINT_DIR, "text_best_model_weights.pt")
eval_model = build_model(CONFIG, ID2LABEL, LABEL2ID)
eval_model.load_state_dict(torch.load(eval_ckpt_path, map_location=device))
eval_model.to(device).eval()

all_preds = []
with torch.no_grad():
    for batch in test_loader:
        logits = eval_model(
            input_ids=batch["input_ids"].to(device),
            attention_mask=batch["attention_mask"].to(device),
        ).logits
        all_preds.extend(logits.argmax(dim=-1).cpu().tolist())

corrected_eval_report(test_df, all_preds, "Text-only")
```

Loading weights: 100%

197/197 [00:00<00:00, 3906.97it/s]

[transformers] **RobertaForSequenceClassification LOAD REPORT** from: roberta-base

Key	Status
-----	--------